01 / DELIVERY

A reliable scheduling core.

Hosts publish availability. Guests reserve a slot. Postgres coordinates every capacity decision.

Repository and running the demo

github.com/Avinaash076/booking-system

Run **docker compose up --build**, then open **http://localhost:3000**. Startup migrates and seeds the database. Host login: **demo / demo-password-change-me**. Change the initial password before deploying publicly.

Public pages: **/demo/discovery** (30-minute, one-on-one) and **/demo/workshop** (60-minute, four seats). Seed hours are Monday-Friday, 09:00-17:00 Asia/Kolkata, with five-minute buffers and four-hour notice.

Delivered scope

Host - Authenticated dashboard, home time zone, event creation, multiple weekly windows, date overrides, upcoming bookings and cancellation reasons.

Guest - Thirty-day slot view, time-zone switcher, five-minute holds, confirmation, private manage link, atomic rescheduling and cancellation.

Reliability - Real Postgres transactions, host row locking, idempotent holds, database-clock expiry, group capacity, buffer conflicts and a transactional notification sink.

Verification and deployment status

14/14 tests passed on Node 22.20.0 and isolated Postgres 17. Production build and Compose configuration validation passed. The Docker engine was unavailable, so the full container stack was not run. Browser verification was blocked by the automation environment. **No public live URL has been deployed.**

02 / ARCHITECTURE

One path to a booking.

A same-origin React client uses an Express JSON API. Postgres owns durable state and coordination.

React UI
Guest / host / manage

>

Express API
Validate + authenticate

Transaction
Lock host + check clock

>

Postgres
Bookings + event sink

Hold request flow

1. Validate - Require an offset-bearing ISO instant and Idempotency-Key. Resolve the requested host and event. Never trust client-side availability.

2. Coordinate - Begin a database transaction. Acquire an advisory transaction lock for the idempotency key, then SELECT the host row FOR UPDATE. All booking and schedule mutations use this host lock.

3. Decide - Read clock_timestamp() after the lock. Expire old holds, load schedule and active occupancy, check notice/horizon, grid alignment, buffers and capacity.

4. Commit - Insert hold plus idempotency record atomically. On confirmation, update the booking and insert its notification event in one transaction. Return the private manage token only after confirmation.

Failure boundaries

A failed transaction rolls back every write. A lost hold response is retried with the same key. A crashed process releases its database locks. Different app instances still coordinate through the same database; no in-memory lock or Redis is involved.

03 / DATA MODEL

Small tables. Clear rules.

Instants are timestamptz. Availability uses local wall-clock minutes plus an IANA zone.

hosts - UUID primary key, unique handle, display name, IANA time zone, salted scrypt password hash. Its row is the lock target for every scheduling mutation.

events - Host foreign key and unique (host_id, slug). Duration 15-120 minutes; capacity 1-100; buffers 0-30; notice 0-43200 minutes; horizon 1-365 days. Database CHECK constraints reinforce input validation.

weekly_windows / date_overrides - Weekly windows use ISO weekday 1-7 and start/end minutes. A per-host/date override replaces all normal hours; an empty JSON array is a day off. The API rejects overlapping windows.

bookings - UUID, host/event, status, start/end, buffered busy bounds, expiry, guest details, private manage token and cancellation reason. A composite foreign key ensures the event belongs to the stored host. CHECK constraints enforce interval order and guest details for confirmed bookings.

idempotency_keys - Globally unique key, normalized-request SHA-256 fingerprint and booking foreign key. A replay returns its original booking; mismatched request data is rejected. Keys have no automatic retention pruning in this core.

notifications - Append-only confirmation, reschedule and cancellation events with booking ID, payload and creation time. This table is the required local sink; it is not a delivery queue with an email provider.

sessions / schema_migrations - SHA-256 session token hashes, host and database expiry; applied migration filenames. Startup migration/seed operations use a database advisory lock.

Where the invariant lives

Primary keys, foreign keys, checks and unique constraints prevent malformed rows. Overlap and group capacity are enforced by the transaction protocol using the locked host row and fresh occupancy reads. Arbitrary SQL writers can bypass that protocol; the database role must remain private to the application.

04 / DECISIONS

Seven deliberate choices.

The priority is a core whose failure behavior is understandable and testable.

1. Host-level row lock - Chosen over event-level locks because different event types share the host. Chosen over per-slot exclusion constraints because group capacity permits some overlaps. Tradeoff: one busy host serializes requests, including slot-list reads.

2. Post-lock database time - Chosen over browser time, process time or transaction-start NOW(). Waiting for a lock can outlive a hold. Expiry decisions use clock_timestamp() after coordination.

3. Live expiry predicate - Chosen over relying on a worker to mark holds expired. Capacity excludes every expired timestamp immediately, even when the stored lifecycle status has not yet been materialized.

4. Request fingerprints - Chosen over client retries that create new reservations. Same key plus same normalized payload returns the original hold; different payload is a 409. Deliberate new intent must use a fresh key.

5. Single atomic reschedule - Chosen over cancel-then-create, which could lose the original booking. Validate the destination excluding the booking itself; update its bounds in the same locked transaction. Destination conflicts roll back.

6. Real instants for time zones - Chosen over adding a fixed offset to weekly hours. Minute-by-minute local membership across real UTC instants correctly handles missing and repeated clock times. Tradeoff: more CPU and longer locks than a tuned interval generator.

7. Transactional sink + seeded auth - Chosen over external email and magic-link infrastructure for the core. Notifications survive process failures with the booking, and seeded cookie sessions keep the host flow self-contained. Production identity and delivery remain follow-up work.

05 / RELIABILITY

The cases that break calendars.

Every capacity-changing API call uses the same ordering and transaction boundary.

A burst of contenders - 40 simultaneous holds produce exactly one winner for capacity one, and exactly three for capacity three. Integration test: "40 simultaneous holds". Other events at the same time cannot overlap the host.

Retries and uncertain responses - 15 same-key hold requests resolve to one hold ID. Changed payload returns 409. Eight confirmation retries emit one event; five cancellation retries emit one cancellation event. Integration tests cover each result and notification row count.

The clock passes during a lock wait - An integration test holds the host lock while a confirmation waits and the hold expires. Confirmation returns 410 after the lock releases. Expired holds free capacity with no worker.

Buffers and group alignment - Busy intervals include before/after buffers. An adjacent event conflicts until those intervals stop overlapping. Only the same event with identical start and end can share group capacity. Tests cover cross-event and adjacent-buffer contention.

A failed or contested move - An occupied destination returns 409 and preserves the original instant. Two bookings racing to a new destination produce one success and one conflict. Successful moves release the old interval.

Forged input and host changes - Invalid instants/zones are rejected; host APIs require a session; hostile browser origins are blocked. Tests exercise login, event creation, weekly hours, day-off overrides, custom override hours, settings, host cancellation and logout.

Residual risks

The public hold path has no distributed rate limit or bot defense. The login throttle is per process. There is no load benchmark, delivery worker, retention policy or production recovery flow. Structured logs avoid tokens and guest payloads, but broader operational controls remain deployment work.

06 / TIME ZONES

Clock time is not elapsed time.

A host schedule is local. A reservation is a pair of UTC instants.

Generation and display

For each host-local date, date overrides replace weekly windows. The generator walks actual UTC minutes between that local midnight and the next, tests each minute against local windows, and merges continuous open intervals. Candidate starts align to a local 15-minute grid. Real duration and both buffers must fit inside an open interval; notice and horizon further filter candidates.

The browser starts with its own IANA zone and offers a switcher. It formats the same returned UTC instants in the selected zone. Labels include the offset/zone abbreviation so a repeated time can be distinguished. The API validates tz; the response remains UTC to preserve unambiguous identity.

Spring forward: New York, March 14, 2027

Before transition - 01:45 EST = 06:45 UTC. The local clock jumps from 01:59 to 03:00.

Thirty real minutes later - The booking ends at 07:15 UTC = 03:15 EDT. No 02:xx start is generated. The test asserts this exact start/end pair and a 30-minute elapsed duration.

Fall back: New York, November 7, 2027

01:30 occurs twice: **05:30 UTC (EDT)** and **06:30 UTC (EST)**. Both may be offered when the host is open. They are separate instants, not duplicate reservations. Unit tests require both UTC values to exist.

Explicit scheduling policies

Availability applies to the entire buffered busy interval. Group sharing requires identical session start/end. Windows do not span midnight; split an overnight pattern into two dates. Host edits affect future selection but keep previously held/confirmed instants intact. The horizon uses elapsed 24-hour days from the database clock. Quarter-hour offset zones are covered by the Kathmandu test.

07 / TESTING AND AI

Evidence, with its limits.

Local results recorded November 6, 2026. No unrun checks are presented as passing.

Results and reproduction

14 tests passed, 0 failed, 0 skipped. Five unit tests cover DST gaps/folds, quarter-hour zones, notice/horizon/buffers, day-off overrides and group/conflict rules. Nine integration tests exercise the HTTP API against actual Postgres, including concurrency and host mutations.

Run **docker compose --profile test run --build --rm tests** for the complete disposable-database suite. Or set **TEST_DATABASE_URL** to a dedicated database and run **npm test**. The suite fails explicitly if that variable is absent. Use **npm run test:unit** for pure tests and **npm run build** for TypeScript and production compilation.

Local verification used Node 22.20.0 and Postgres 17 on an isolated localhost port. Build passed; Compose configuration parsed successfully. GitHub Actions is configured for Node 22/Postgres 16 on pushes and pull requests; its remote result must be checked after pushing. Docker runtime and browser end-to-end checks were not run successfully.

AI-assisted workflow

OpenAI Codex extracted the supplied brief, inspected the empty repository, designed the lock protocol, and wrote the application, tests and documentation. The user supplied the task and destination. No user-authored manual code edits were observed. AI_LOG.md contains the working priorities, corrections and verification record.

Corrections included an unavailable PDF library (replaced with Poppler extraction), Windows TLS backend failure (OpenSSL with verification enabled), sandbox write restrictions, and an unavailable Docker engine (replaced for local testing with isolated Postgres). Code inspection prompted additional host settings and stale-clock/lock-wait tests. Browser automation was blocked and is listed as a gap.

References used

PostgreSQL 17: Explicit Locking
Luxon API documentation

08 / HANDOFF

What to do next.

Review the core, run the demo, then choose how far to take deployment and optional work.

Optional work included

A GitHub Actions build/test workflow, structured logs with request IDs, a database-backed health endpoint, and a notification sink viewer. Native labels, buttons and forms support keyboard interaction, but no complete accessibility audit is claimed. Observability is partial: no notification-lag or booking metrics dashboard is included.

Known gaps

No live deployment, real email, calendar sync or invites, reminders, webhooks, waitlist, round-robin routing, load test, browser E2E suite, distributed hold-abuse protection or production account recovery. Event types can be created but not edited/deleted in this core. The host UI time control supports up to 23:59; the API supports 1440 as an exclusive end minute.

With another day

First - Run guest and host browser flows at desktop/mobile widths, add Playwright coverage, test a full Docker cold start, and check remote CI. Add shared public rate limits and anti-abuse controls before exposing holds.

Then - Profile slot generation and lock duration, add metrics and load results, narrow the database role, implement backup/restore and retention, and replace seeded credentials with production account management.

Deploy - Use the Dockerfile with managed Postgres 15+, an HTTPS APP_URL, DATABASE_URL, PORT and a strong initial DEMO_PASSWORD. Configure provider TLS and private DB networking. Record the real live URL only after deployment and browser verification.

Submission checklist

Review and personalize this AI-authored report. Keep the unsquashed commit history. Ensure the public GitHub repository contains source, migrations, seed, README, AI_LOG and tests. Add a live URL only if deployed. Submit the repository link and this PDF yourself to the recruiter; no recruiter communication has been sent.